Project Overview

This project investigates how regression model complexity and regularization affect prediction performance using the **UCI Concrete Compressive Strength Dataset**.

Linear Regression, Polynomial Regression, Ridge Regression, and Lasso Regression are implemented and evaluated using Python and Scikit-learn. The models are compared using MSE, RMSE, and R^2 to understand underfitting, overfitting, and generalization.

Objectives

- Load and clean a real-world dataset from the UCI Machine Learning Repository.
- Perform exploratory data analysis and visualization.
- Train a baseline Linear Regression model.
- Apply Polynomial Regression with multiple degrees.
- Identify potential underfitting and overfitting.
- Apply Ridge and Lasso regularization.
- Compare model performance using appropriate regression metrics.
- Visualize actual versus predicted values.

Dataset

- **Name:** Concrete Compressive Strength
- **Source:** UCI Machine Learning Repository
- **Number of records:** 1,030 (before cleaning)
- **Input features:** 8
- **Target variable:** Concrete compressive strength (MPa)

Input Features

1. Cement
2. Blast furnace slag
3. Fly ash
4. Water
5. Superplasticizer
6. Coarse aggregate
7. Fine aggregate

8. Age

Target: Concrete compressive strength, measured in megapascals (MPa).

Technologies Used

- Python
- Google Colab
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Machine Learning Models

1. Linear Regression

Used as the baseline model to predict concrete compressive strength from the input features.

2. Polynomial Regression

Polynomial features are generated using degrees 1, 2, 3, 4, and 5 to investigate how increasing model complexity affects prediction performance.

3. Ridge Regression

Applies L2 regularization to shrink model coefficients and help control model complexity.

4. Lasso Regression

Applies L1 regularization, which can shrink some coefficients to zero and perform implicit feature selection.

Project Workflow

1. Import the required libraries.
2. Download and load the UCI dataset.
3. Explore the dataset and inspect missing values and duplicates.
4. Clean inconsistent or invalid values.
5. Visualize feature relationships and target distribution.

6. Separate the input features and target variable.
7. Split the data into training and testing sets.
8. Train and evaluate the baseline Linear Regression model.
9. Train Polynomial Regression models with multiple degrees.
10. Compare training and testing performance.
11. Apply Ridge and Lasso regularization.
12. Compare all models using a performance table.
13. Visualize actual versus predicted values.
14. Summarize the results and conclusions.

Evaluation Metrics

Mean Squared Error (MSE)

Measures the average squared difference between actual and predicted values. Lower values indicate smaller prediction errors.

Root Mean Squared Error (RMSE)

The square root of MSE. It measures prediction error in the same unit as the target variable, MPa. Lower values are better.

R^2 Score

Measures how much of the target's variance is explained by the model. Higher values generally indicate better predictive performance.

Underfitting and Overfitting

- **Underfitting:** The model is too simple to capture important patterns, resulting in poor training and testing performance.
- **Overfitting:** The model fits training data well but performs substantially worse on unseen test data.
- **Generalization:** A model performs well on testing data, with a reasonably small gap between training and testing performance.

These patterns are assessed using the actual evaluation results rather than assuming that a particular polynomial degree must overfit.

Regularization Impact

- Ridge Regression penalizes the squared magnitude of coefficients.

- Lasso Regression penalizes the absolute magnitude of coefficients and can set some coefficients to zero.
- The alpha parameter controls regularization strength.
- Regularization may improve generalization, but its effectiveness depends on the dataset and parameter values.

Results and Visualizations

The notebook generates:

- Cement content versus concrete strength scatter plot.
- Concrete strength distribution histogram.
- Polynomial degree versus testing RMSE graph.
- Training versus testing R^2 graph.
- Model comparison table containing MSE, RMSE, and R^2.
- Actual versus predicted strength scatter plot.

The numerical results should be taken from the executed notebook.

How to Run the Project

1. Open Google Colab.
2. Upload or open Assignment_2_Nonlinear_Regression.ipynb.
3. Run all cells sequentially.
4. If prompted, install the required Excel-reading dependency using %pip install xlrd.
5. Review the evaluation tables and generated graphs.
6. Save or download the completed notebook.

No local Python installation is required when using Google Colab.

Conclusion

This project compares linear and nonlinear regression approaches for predicting concrete compressive strength. Polynomial Regression is used to study the effect of model complexity, while Ridge and Lasso are used to investigate regularization.

MSE, RMSE, and R^2 provide complementary measures of model performance. Comparing training and testing results helps identify potential underfitting and

overfitting. The final model is selected based on the observed test results and generalization performance.

Author

Machine Learning Assignment – 2